

Reviewer Pack — Real Dimension Gap in Communication Matrices for AC⁰ PARITY

Entry 54a84a6f394b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 23:16:50 UTC

Real Dimension Gap in Communication Matrices for AC⁰ PARITY

Entry ID: 54a84a6f394b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 23:16:50 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): Communication Complexity

Statement:

For any AC⁰ circuit C computing the parity function on n bits, the real dimension of the solution set to the system of equations defined by the communication matrix of C is $\Omega(\sqrt{n})$.

Rationale (proposer’s reasoning):

The real dimension of the solution set captures the geometric complexity of the communication matrix, which may be tied to the circuit’s size. High real dimension could imply that the function requires larger circuits.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0fb4ce013e00775f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def rank(A):
    rref = gaussian_elimination(A)
    rank = 0
    for row in rref:
        if any(row):
            rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
        # Generate a random CNF instance for parity
        cnf = []
        for _ in range(n):
            clause = [random.randint(1, n) * (-1 if random.choice([True, False]) else 1)]
            cnf.append(clause)

        # Compute the communication matrix
        A = [[0] * (2**n) for _ in range(2**n)]
```

```

    for i in range(2**n):
        for j in range(2**n):
            if all((i & (1 << abs(l)-1)) == (j & (1 << abs(l)-1)) or l < 0 for l in cnf[i]):
                A[i][j] = 1

    # Approximate the real dimension of the solution set
    dim = rank(A)

    total_metric_value += dim
    instances_tested += len(n_values)

metric_value = total_metric_value / instances_tested

conjecture_holds = math.sqrt(n) <= metric_value for n in n_values
if not all(conjecture_holds):
    counterexample = "dim does not scale with sqrt(n)"
else:
    counterexample = ""

return {
    "metric_name": "Real Dimension",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": all(conjecture_holds),
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 10**9) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSEIFIED counterexample=\"dim does not scale with sqrt(n)\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dla3e697.py", line 107, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dla3e697.py", line 76, in run_trial
    dim = rank(A)
          ^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dla3e697.py", line 52, in rank
    rref = gaussian_elimination(A)
           ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dla3e697.py", line 25, in gaussian_elim
    factor = A[j][i] / A[i][i]
             ~~~~~^~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with division-by-zero error, preventing dimension calculation. The crash indicates a potential flaw in the matrix construction or Gaussian elimination implementation. | next: Debug the Gaussian elimination code to handle singular matrices, then re-run with stabilized numerical methods

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	95648
2	preregistration	ollama_remote	qwen3:8b	0	0	24249
3	novelty	ollama_remote	qwen3:8b	0	0	21011
4	novelty	ollama_remote	qwen3:8b	0	0	13605
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14583
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9851
7	judge	ollama_remote	qwen3:8b	0	0	19981

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 198928 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/54a84a6f394b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/54a84a6f394b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/54a84a6f394b.tar.gz` (if generated)